

Planification d'itinéraires cyclistes

Cahier des charges

Alexandre Genin, Ron Sharony, Alex Thirault

2026

Table des matières

1	Introduction	1
2	Déroulement du projet	1
3	État de l'art	1
4	Prérequis et concepts de base	2
5	Approche algorithmique	3
6	Implémentation	3
7	Fonctionnalités de l'application	4

1 Introduction

Les applications de planification d'itinéraires sont des services particulièrement courants de nos jours. Avec l'essor des déplacements à vélo, diverses applications spécialement dédiées aux cycles se sont développées, parmi lesquelles Geovelo, ou encore Naviki.

L'objet de ce projet sera de créer un nouvel outil de planification d'un itinéraire cycliste qui tient compte du niveau de sécurité des voies empruntées (lié par exemple à la présence d'une piste cyclable, à l'absence de carrefour mal protégé, etc.) : c'est une caractéristique ordinale. Bien que ce critère de choix soit proposé dans certains outils déjà existants, l'approche développée reposera sur une approche originale, avec une modélisation multi-objectifs du problème, où chaque objectif représentera le nombre de kilomètres du trajet respectant un certain niveau de sécurité. L'application développée retournera un ensemble d'itinéraires possibles, réalisant différents compromis temps de trajet/sécurité.

On s'appuiera sur les données réelles d'OpenStreetMap pour réaliser l'application qui fonctionnera en local sur une machine.

2 Déroulement du projet

Ci-dessous les démarches que nous prenons afin de réaliser ce projet :

- appropriation du sujet,
- découverte de ce qui a déjà été fait sur cette même problématique par Geovelo et Naviki,
- choix de l'algorithme utilisé pour résoudre le problème,
- implémentation en Python de l'algorithme,
- utilisation de l'algorithme sur des données réelles issues de OpenStreetMap,
- conception d'une application conviviale fonctionnant en local permettant de déterminer les itinéraires possibles en fonction de compris temps/sécurité,
- comparaison de nos itinéraires avec ceux des applications déjà existantes.

3 État de l'art

Présentons ce qui est déjà implémenté chez Geovelo.

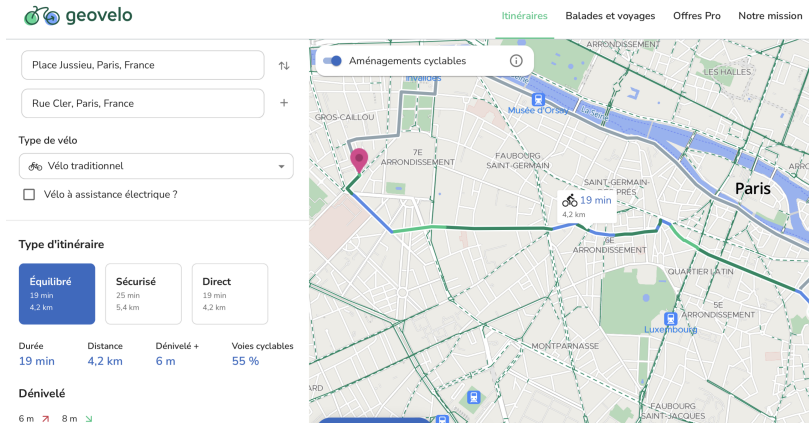


FIGURE 1 – Itinéraire proposé sur Geovelo entre la Place Jussieu et la Rue Cler

L'utilisateur choisit une adresse de départ, une adresse de destination et Geovelo propose alors trois itinéraires distincts : un équilibré, un sécurisé et un direct. Quant aux critères sur les itinéraires, Geovelo mentionne :

- les pistes cyclables et voies dédiées (voies comportant un aménagement cyclable séparé de la circulation automobile),
- les double-sens cyclables,
- les voies cyclables partagées (voies comportant un aménagement cyclable partagé avec la circulation automobile (bandes, voies de bus)),
- les voies au trafic apaisé (voies avec une circulation automobile inférieure à 30km/h ou voies partagée avec les piétons),
- les zones temporairement bloquées aux cyclistes (les cyclistes ne pourront pas circuler dans ces zones et seront invités à emprunter des itinéraires de contournement),
- et enfin, les principaux axes cyclables (axes permettent de relier les principaux pôles d'activité du territoire, avec des aménagements adaptés aux cyclistes.).

Bien sûr, la durée, la distance, le dénivelé de chacun des trajets sont mentionnés. Les voies empruntées par l'itinéraire le sont aussi. Notons que l'application offre le choix de préciser le type de vélo utilisé.

Naviki offre moins de liberté à l'utilisateur en proposant un seul trajet similaire au trajet équilibré de Geovelo.

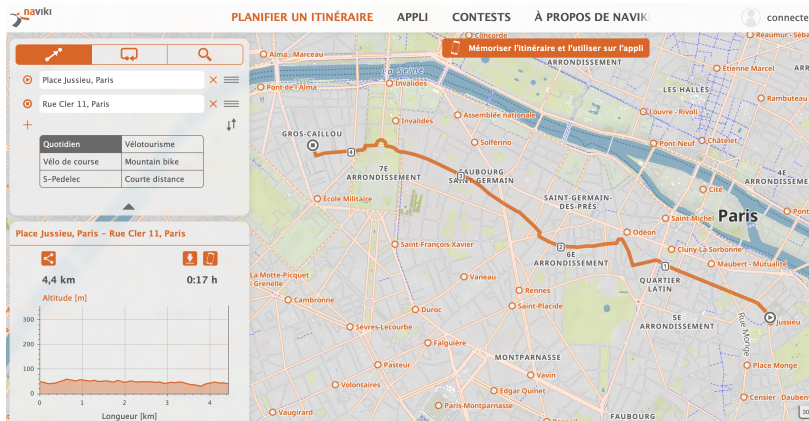


FIGURE 2 – Itinéraire proposé sur Naviki entre la Place Jussieu et la Rue Cler

4 Prérequis et concepts de base

Il existe une reformulation multi-objectifs de notre problème d'optimisation ordinaire, et nous explicitons donc les outils usuels dont nous allons avoir besoin.

On note $G = (V, A)$ un graphe orienté G où V est l'ensemble de ses sommets et A l'ensemble de ses arcs. Chaque arc est muni d'un vecteur de coûts représentant les coûts de chaque objectif. Un chemin est représenté par son vecteur de coûts selon tous les objectifs et par la liste des sommets dont il est constitué. Un label L est alors défini par $L = [v, \bar{c}, \text{Prev}_L]$ où v est le sommet auquel le label est associé, $\bar{c}$ le vecteur de coûts du chemin arrivant en v passant avant par le sommet de label Prev_L . On introduit désormais les relations dont on a besoin afin de comparer les labels.

Définition 1. (*Dominance*)

Le vecteur $[a_1, \dots, a_n]$ domine le vecteur $[b_1, \dots, b_n]$ si et seulement si :

$$(\forall i \in \{1, \dots, n\} : a_i \leq b_i) \wedge (\exists i \in \{1, \dots, n\} : a_i < b_i)$$

- Un chemin P_1 domine un autre chemin P_2 si et seulement si le vecteur de coûts de P_1 domine celui de P_2 .
- Un label L_1 domine un autre label L_2 si et seulement si le vecteur définissant L_1 domine celui de L_2 .

Définition 2. (*Optimalité au sens de Pareto*)

Un chemin est dit Pareto optimal si et seulement s'il n'existe aucun autre chemin réalisable qui soit meilleur pour l'un des objectifs et non moins bon pour tous les autres objectifs.

Un ensemble de chemins S est dit Pareto optimal si et seulement si aucun des éléments de S n'est dominé par un autre chemin réalisable.

À plusieurs reprises nous aurons besoin d'un ordre sur les labels afin de faire un choix sur ces derniers. Dans le but de déterminer le label le plus petit ou le plus grand, on se sert de l'ordre lexicographique ainsi défini :

Définition 3. (*Ordre lexicographique*)

Le vecteur $[a_1, \dots, a_n]$ est lexicographiquement plus petit que (noté $<_l$) le vecteur $[b_1, \dots, b_n]$ si et seulement si :

$$\exists k \in \{1, \dots, n\} : (\forall i < k : a_i = b_i) \wedge (a_k < b_k)$$

Remarque 1. Cette relation est une relation d'ordre totale, car elle est réflexive, antisymétrique et transitive, et deux éléments peuvent toujours être comparés entre eux.

L'ordre lexicographique n'est pas forcément le meilleur ordonnancement pour les algorithmes de plus court chemin multi-objectifs. Dans l'étude de Paixão et Santos (2007), il est démontré qu'un ordonnancement par agrégat de somme pondérée permet une réduction non négligeable du temps de calcul. Par soucis de facilité, on se sert quand même de l'ordre lexicographique.

5 Approche algorithmique

Martins présente en 1984 un algorithme, basé sur celui de Dijkstra, de marquage de labels afin de traiter le problème du plus court chemin multi-objectifs où est associé à chaque sommet un ensemble de labels. Chaque ensemble n'est constitué que de labels non dominés. Ainsi, on n'ajoute un nouveau label à un sommet que si celui-ci n'est pas dominé par un des labels déjà existants. De même, les labels dominés par celui que l'on ajoute sont supprimés : cela permet de ne pas avoir une explosion du nombre de chemins que l'on étudie. Mais cet algorithme s'avère être trop lent pour de grands réseaux comme les réseaux de transports, et ainsi, trop lent pour notre problème de planification d'itinéraires cyclistes.

Pour pallier cet exact problème, Demeyer, Goedgebeur, Audenaert, Pickavet et Demeester présentent dans *Speeding up Martins' algorithm for multiple objective shortest path problems* un algorithme bidirectionnel, où la procédure usuelle de recherche est séparée en une procédure avant (partant du sommet source) et une procédure arrière (partant du sommet destination), pour le problème du plus court chemin multi-objectifs. On note alors que chaque sommet a désormais deux ensembles de labels : un ensemble avant et un ensemble arrière.

L'efficacité des algorithmes bidirectionnels pour les problèmes de plus courts chemins dépend de la manière dont on alterne entre la recherche avant et arrière ainsi que, principalement, de la condition d'arrêt. Pour un ensemble T de labels, on pose $\min(T) = [\min_1(T), \dots, \min_n(T)]$ le vecteur correspondant au minimum des coûts sur tous les labels de T pour chacun des n objectifs. On comparera les chemins de l'ensemble des chemins solutions au vecteur $\min(T_{\text{avant}}) + \min(T_{\text{arrière}})$.

Proposition 5.1. Les recherches avant et arrière peuvent être arrêtées dès lors que $\min(T_{\text{avant}}) + \min(T_{\text{arrière}})$ est dominé par un chemin de l'ensemble des chemins solutions.

6 Implémentation

On a ainsi le pseudo-code suivant :

- 1: $T[\text{avant}] \leftarrow \emptyset$
- 2: $T[\text{arrière}] \leftarrow \emptyset$

```

3:  $L_{\text{résultat}} \leftarrow \emptyset$ 

4:  $labelSource \leftarrow [source, [0, \dots, 0], \text{NULL}]$ 
5:  $source.\text{ajouterLabel}(labelSource, \text{avant})$ 
6:  $T[\text{avant}].\text{ajouter}(labelSource)$ 

7:  $labelDest \leftarrow [destination, [0, \dots, 0], \text{NULL}]$ 
8:  $destination.\text{ajouterLabel}(labelDest, \text{arrière})$ 
9:  $T[\text{arrière}].\text{ajouter}(labelDest)$ 

10: while  $\min(T_{\text{avant}}) + \min(T_{\text{arrière}})$  n'est dominé par aucun  $R \in L_{\text{résultat}}$  do
11:    $d \leftarrow \text{obtenirDirection}()$ 
12:    $label \leftarrow T[d].\text{extraireMin}()$ 
13:    $proprietaire \leftarrow label.\text{proprietaire}()$ 
14:    $voisins \leftarrow proprietaire.\text{voisins}()$ 
15:   for all  $n \in voisins$  do
16:      $lien \leftarrow \text{obtenirLienEntre}(proprietaire, n)$ 
17:      $nouveauLabel \leftarrow [n, [label[i] + lien.cout[i]], label]$ 
18:     if  $nouveauLabel$  n'est dominé par aucun label de  $n.\text{labels}(d)$  then
19:        $n.\text{ajouterLabel}(nouveauLabel, d)$ 
20:        $T[d].\text{ajouter}(nouveauLabel)$ 
21:       if  $n.\text{label}(d')$  n'est pas vide then
22:          $resultats \leftarrow \text{combiner}(nouveauLabel, n.\text{labels}(d'))$ 
23:          $L_{\text{résultat}}.\text{ajouterResultats}(resultats)$ 
24:       end if
25:     end if
26:   end for
27: end while

```

7 Fonctionnalités de l'application

L'application permettra d'obtenir plusieurs itinéraires cyclistes entre une localisation de départ et une autre d'arrivée toutes deux choisies par l'utilisateur. Ce dernier pourra spécifier ses préférences concernant les critères ordinaux du problème pour obtenir les trajets qui lui conviendraient le mieux.